SQL Cheatsheet

Từ cơ bản đến nâng cao · cú pháp · ví dụ · kết quả

Mọi kết quả trong tài liệu được **chạy thực tế trên SQLite 3.45** với bộ dữ liệu mẫu ở mục “Dữ liệu mẫu”. Cú pháp bám theo chuẩn SQL; chỗ nào khác biệt giữa MySQL, PostgreSQL, SQL Server, Oracle đều có ghi chú, và có bảng so sánh ở phụ lục.

Mục lục

Dữ liệu mẫu 2

Tra cứu nhanh 4

1. Truy vấn cơ bản (SELECT) 5

2. Hàm tổng hợp & GROUP BY 8

3. Tạo & thay đổi dữ liệu (DDL / DML) 9

4. JOIN – ghép nhiều bảng 12

5. Truy vấn con (Subquery) 15

6. Phép toán tập hợp 16

7. Hàm & biểu thức 18

8. Nâng cao 20

9. Mẫu truy vấn hay gặp 26

10. Phụ lục 28

Dữ liệu mẫu

Các ví dụ trong tài liệu dùng chung 4 bảng nhỏ dưới đây (nhân viên – phòng ban – sản phẩm – đơn hàng). Chú ý có sẵn các trường hợp đặc biệt: nhân viên **Khoa** chưa thuộc phòng nào, phòng **Marketing** chưa có nhân viên, và đơn hàng ở nhiều trạng thái.

CẤU TRÚC BẢNG

```
CREATE TABLE departments (dept_id INTEGER PRIMARY KEY, dept_name TEXT, location TEXT);
CREATE TABLE employees (
  emp_id INTEGER PRIMARY KEY, name TEXT, dept_id INTEGER, manager_id INTEGER,
  salary INTEGER, hire_date TEXT, job_title TEXT
);
CREATE TABLE products (
  product_id INTEGER PRIMARY KEY, product_name TEXT, category TEXT, price INTEGER
);
CREATE TABLE orders (
  order_id INTEGER PRIMARY KEY, customer TEXT, product_id INTEGER,
  qty INTEGER, order_date TEXT, status TEXT
);
```

BẢNG DEPARTMENTS · 4 dòng

dept_id	dept_name	location
1	IT	Hà Nội
2	HR	TP.HCM
3	Sales	Đà Nẵng
4	Marketing	Hà Nội

BẢNG EMPLOYEES · 9 dòng

emp_id	name	dept_id	manager_id	salary	hire_date	job_title
1	An	1	*NULL*	3000	2019-03-01	Director
2	Bình	1	1	2200	2020-06-15	Developer
3	Chi	1	1	2000	2021-01-10	Developer
4	Dũng	2	1	1800	2020-09-01	HR Manager
5	Em	3	1	2500	2018-11-20	Sales Lead
6	Phúc	3	5	1500	2022-02-14	Sales Rep
7	Giang	3	5	1600	2022-07-01	Sales Rep
8	Hà	2	4	1400	2023-01-05	HR Staff
9	Khoa	*NULL*	1	1700	2023-05-20	Intern

BẢNG PRODUCTS · 5 dòng

product_id	product_name	category	price
1	Laptop	Electronics	1200
2	Mouse	Electronics	25
3	Desk	Furniture	300
4	Chair	Furniture	150
5	Monitor	Electronics	400

BẢNG ORDERS · 10 dòng

order_id	customer	product_id	qty	order_date	status
1	Alice	1	1	2026-01-05	done
2	Bob	2	3	2026-01-12	done
3	Alice	5	2	2026-02-03	done
4	Carol	3	1	2026-02-14	pending
5	Bob	1	1	2026-03-01	done
6	Dave	4	4	2026-03-09	cancelled
7	Carol	2	10	2026-03-20	done
8	Alice	4	2	2026-04-02	pending
9	Eve	5	1	2026-04-18	done
10	Bob	3	2	2026-05-07	done

Tra cứu nhanh

Nhóm	Cú pháp	Công dụng
Truy vấn	`SELECT cols FROM t WHERE cond`	Lấy dữ liệu có điều kiện
	`SELECT DISTINCT col FROM t`	Loại bỏ giá trị trùng
	`ORDER BY col [ASC|DESC]`	Sắp xếp kết quả
	`LIMIT n OFFSET m`	Giới hạn dòng / phân trang
Điều kiện	`= <> < > <= >= AND OR NOT`	So sánh và logic
	`IN (...) BETWEEN a AND b LIKE 'a%' IS NULL`	Tập giá trị, khoảng, mẫu chuỗi, NULL
Tổng hợp	`COUNT SUM AVG MIN MAX`	Hàm tổng hợp
	`GROUP BY col HAVING cond`	Nhóm dòng; lọc theo nhóm
JOIN	`a INNER JOIN b ON a.k = b.k`	Chỉ giữ dòng khớp ở cả 2 bảng
	`a LEFT JOIN b ON ...`	Giữ mọi dòng của bảng trái
	`a RIGHT JOIN b` / `a FULL JOIN b`	Giữ mọi dòng bảng phải / cả hai bảng
	`a CROSS JOIN b`	Tích Descartes (mọi cặp dòng)
Tập hợp	`UNION [ALL] INTERSECT EXCEPT`	Hợp / giao / hiệu hai kết quả
Truy vấn con	`WHERE x IN (SELECT ...) EXISTS (...)`	Subquery trong WHERE / FROM / SELECT
DML	`INSERT INTO t (cols) VALUES (...)`	Thêm dòng
	`UPDATE t SET col = v WHERE cond`	Sửa dòng
	`DELETE FROM t WHERE cond`	Xóa dòng
DDL	`CREATE TABLE ALTER TABLE DROP TABLE`	Tạo / sửa / xóa bảng
	`CREATE INDEX idx ON t (col)`	Chỉ mục tăng tốc truy vấn
	`CREATE VIEW v AS SELECT ...`	Bảng ảo lưu câu truy vấn
Hàm	`UPPER LOWER LENGTH SUBSTR TRIM REPLACE`	Hàm chuỗi
	`CASE WHEN c THEN a ELSE b END`	Rẽ nhánh trong câu lệnh
	`COALESCE(a,b) NULLIF(a,b) CAST(x AS t)`	Xử lý NULL, ép kiểu
Nâng cao	`WITH cte AS (SELECT ...) SELECT ...`	CTE (truy vấn tạm có tên)
	`fn() OVER (PARTITION BY a ORDER BY b)`	Window function
	`BEGIN; ... COMMIT;` / `ROLLBACK;`	Giao dịch (transaction)

1. Truy vấn cơ bản (SELECT)

1.1 SELECT … FROM

Lấy dữ liệu từ bảng. `*` là mọi cột; nên liệt kê cột cần dùng thay vì `*` để rõ ràng và nhanh hơn.

CÚ PHÁP

```sql
SELECT column1, column2, ...
FROM table_name;
```

VÍ DỤ

```sql
SELECT dept_name, location
FROM departments;
```

KẾT QUẢ · 4 dòng

dept_name	location
IT	Hà Nội
HR	TP.HCM
Sales	Đà Nẵng
Marketing	Hà Nội

1.2 Bí danh (AS) và biểu thức

Đặt tên cho cột/biểu thức bằng `AS`. Có thể tính toán ngay trong SELECT.

CÚ PHÁP

```sql
SELECT expression AS alias
FROM table_name;
```

VÍ DỤ

```sql
SELECT name AS ten, salary * 12 AS luong_nam
FROM employees
LIMIT 3;
```

KẾT QUẢ · 3 dòng

ten	luong_nam
An	36000
Bình	26400
Chi	24000

1.3 WHERE – lọc dòng

Chỉ giữ các dòng thỏa điều kiện. Kết hợp bằng `AND`, `OR`, `NOT`; so sánh bằng `= <> < > <= >=`.

CÚ PHÁP

```sql
SELECT columns
FROM table_name
WHERE condition1 AND condition2;
```

VÍ DỤ

```sql
SELECT name, salary
FROM employees
WHERE dept_id = 1 AND salary >= 2000;
```

KẾT QUẢ · 3 dòng

name	salary
An	3000
Bình	2200
Chi	2000

1.4 IN và BETWEEN

`IN` kiểm tra thuộc một tập giá trị. `BETWEEN a AND b` lấy khoảng, **bao gồm cả hai đầu mút**.

CÚ PHÁP

```
WHERE col IN (v1, v2, ...)
WHERE col BETWEEN low AND high
```

VÍ DỤ

```
SELECT name, dept_id, salary
FROM employees
WHERE dept_id IN (2, 3) AND salary BETWEEN 1400 AND 1600;
```

KẾT QUẢ · 3 dòng

name	dept_id	salary
Phúc	3	1500
Giang	3	1600
Hà	2	1400

1.5 LIKE – tìm theo mẫu chuỗi

`%` khớp 0 hoặc nhiều ký tự, `_` khớp đúng 1 ký tự.

CÚ PHÁP

```
WHERE col LIKE 'pattern'
```

VÍ DỤ

```
SELECT name, job_title
FROM employees
WHERE job_title LIKE 'Sales%' OR name LIKE 'D_ng';
```

KẾT QUẢ · 4 dòng

name	job_title
Dũng	HR Manager
Em	Sales Lead
Phúc	Sales Rep
Giang	Sales Rep

Lưu ý: Phân biệt hoa/thường tùy hệ CSDL: PostgreSQL có `ILIKE` để không phân biệt; MySQL mặc định không phân biệt.

1.6 IS NULL / IS NOT NULL

NULL nghĩa là “không có giá trị”. Không dùng `= NULL`; phải dùng `IS NULL`.

CÚ PHÁP

```
WHERE col IS NULL
WHERE col IS NOT NULL
```

VÍ DỤ

```
SELECT name, job_title
FROM employees
WHERE dept_id IS NULL;
```

KẾT QUẢ · 1 dòng

name	job_title
Khoa	Intern

1.7 ORDER BY, LIMIT, OFFSET

Sắp xếp (`ASC` tăng – mặc định, `DESC` giảm) rồi lấy một phần kết quả. `OFFSET` bỏ qua n dòng đầu (dùng để phân trang).

CÚ PHÁP

```
SELECT columns FROM table_name
ORDER BY col1 [ASC|DESC], col2 [ASC|DESC]
LIMIT n OFFSET m;
```

VÍ DỤ

```
SELECT name, salary
FROM employees
ORDER BY salary DESC, name ASC
LIMIT 3 OFFSET 1;
```

KẾT QUẢ · 3 dòng

name	salary
Em	2500
Bình	2200
Chi	2000

1.8 DISTINCT – bỏ giá trị trùng

Trả về các giá trị khác nhau (NULL được tính là một giá trị).

CÚ PHÁP

```
SELECT DISTINCT col1, col2 FROM table_name;
```

VÍ DỤ

```
SELECT DISTINCT dept_id
FROM employees
ORDER BY dept_id;
```

KẾT QUẢ · 4 dòng

dept_id
NULL
1
2
3

2. Hàm tổng hợp & GROUP BY

2.1 COUNT, SUM, AVG, MIN, MAX

Gom nhiều dòng thành một giá trị. `COUNT(*)` đếm mọi dòng; `COUNT(col)` và các hàm còn lại **bỏ qua NULL**.

CÚ PHÁP

```sql
SELECT COUNT(*), COUNT(col), SUM(col), AVG(col), MIN(col), MAX(col)
FROM table_name;
```

VÍ DỤ

```sql
SELECT COUNT(*) AS so_nv, COUNT(dept_id) AS co_phong,
       SUM(salary) AS tong_luong, ROUND(AVG(salary), 1) AS luong_tb,
       MIN(salary) AS thap_nhat, MAX(salary) AS cao_nhat
FROM employees;
```

KẾT QUẢ · 1 dòng

so_nv	co_phong	tong_luong	luong_tb	thap_nhat	cao_nhat
9	8	17700	1966.7	1400	3000

2.2 GROUP BY – nhóm dữ liệu

Chia dòng thành các nhóm theo cột rồi tính hàm tổng hợp cho từng nhóm. Mọi cột trong SELECT hoặc phải nằm trong `GROUP BY`, hoặc phải nằm trong hàm tổng hợp.

CÚ PHÁP

```sql
SELECT group_col, AGG(col)
FROM table_name
GROUP BY group_col;
```

VÍ DỤ

```sql
SELECT dept_id, COUNT(*) AS so_nv, ROUND(AVG(salary), 1) AS luong_tb
FROM employees
GROUP BY dept_id;
```

KẾT QUẢ · 4 dòng

dept_id	so_nv	luong_tb
NULL	1	1700.0
1	3	2400.0
2	2	1600.0
3	3	1866.7

2.3 HAVING – lọc nhóm

`WHERE` lọc **dòng** trước khi nhóm; `HAVING` lọc **nhóm** sau khi đã tổng hợp.

CÚ PHÁP

```sql
SELECT group_col, AGG(col)
FROM table_name
WHERE row_condition
GROUP BY group_col
HAVING group_condition;
```

VÍ DỤ

```sql
SELECT dept_id, COUNT(*) AS so_nv, SUM(salary) AS tong_luong
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id
HAVING COUNT(*) >= 3;
```

KẾT QUẢ · 2 dòng

dept_id	so_nv	tong_luong
1	3	7200
3	3	5600

2.4 COUNT(DISTINCT …)

Đếm số giá trị khác nhau.

CÚ PHÁP

```
SELECT COUNT(DISTINCT col) FROM table_name;
```

VÍ DỤ

```
SELECT COUNT(DISTINCT customer) AS so_khach, COUNT(*) AS so_don
FROM orders;
```

KẾT QUẢ · 1 dòng

so_khach	so_don
5	10

2.5 Thứ tự xử lý của một câu SELECT

Viết theo thứ tự SELECT → FROM, nhưng CSDL xử lý theo thứ tự dưới đây. Vì vậy alias khai báo ở SELECT dùng được trong `ORDER BY` nhưng **không** dùng được trong `WHERE`.

Bước	Mệnh đề	Việc làm
1	`FROM` / `JOIN`	Xác định bảng nguồn và ghép bảng
2	`WHERE`	Lọc từng dòng
3	`GROUP BY`	Gom dòng thành nhóm
4	`HAVING`	Lọc nhóm
5	`SELECT`	Tính cột / biểu thức / alias
6	`DISTINCT`	Loại dòng trùng
7	`ORDER BY`	Sắp xếp
8	`LIMIT` / `OFFSET`	Cắt kết quả

3. Tạo & thay đổi dữ liệu (DDL / DML)

3.1 Kiểu dữ liệu thường gặp

Tên kiểu có thể khác nhau chút ít giữa các hệ CSDL; SQLite dùng “dynamic typing” nên khá linh hoạt.

Nhóm	Kiểu phổ biến	Ghi chú
Số nguyên	`INT` / `INTEGER`, `SMALLINT`, `BIGINT`	
Số thập phân	`DECIMAL(p,s)` / `NUMERIC`, `FLOAT`, `REAL`	Tiền tệ nên dùng `DECIMAL`
Chuỗi	`CHAR(n)`, `VARCHAR(n)`, `TEXT`	
Ngày giờ	`DATE`, `TIME`, `TIMESTAMP`	SQLite lưu dạng chuỗi ISO `YYYY-MM-DD`
Logic	`BOOLEAN`	MySQL: `TINYINT(1)`
Khác	`BLOB`, `JSON`	

3.2 CREATE TABLE và ràng buộc

Tạo bảng mới cùng các ràng buộc (constraint) để bảo vệ tính toàn vẹn dữ liệu.

CÚ PHÁP

```
CREATE TABLE table_name (
  column1 datatype [constraint],
  column2 datatype [constraint],
  ...
);
```

VÍ DỤ

```
CREATE TABLE projects (
  project_id INTEGER PRIMARY KEY,
  title      TEXT NOT NULL UNIQUE,
  budget     INTEGER CHECK (budget >= 0),
  status     TEXT DEFAULT 'new',
  dept_id    INTEGER REFERENCES departments(dept_id)
);
```

✓ Thực thi thành công

3.3 Các loại ràng buộc

Ràng buộc	Ý nghĩa
`PRIMARY KEY`	Khóa chính: duy nhất và không NULL, định danh mỗi dòng
`FOREIGN KEY` / `REFERENCES`	Khóa ngoại: giá trị phải tồn tại ở bảng được tham chiếu
`UNIQUE`	Không cho phép giá trị trùng trong cột
`NOT NULL`	Không cho phép NULL
`CHECK (cond)`	Giá trị phải thỏa điều kiện
`DEFAULT v`	Giá trị mặc định khi không chỉ định lúc INSERT

3.4 INSERT – thêm dữ liệu

Chỉ rõ danh sách cột để an toàn khi bảng thay đổi cấu trúc. Có thể thêm nhiều dòng trong một lệnh.

CÚ PHÁP

```
INSERT INTO table_name (col1, col2, ...)
VALUES (v1, v2, ...), (v1, v2, ...);

INSERT INTO table_name (col1, col2)
SELECT colA, colB FROM other_table;
```

VÍ DỤ

```
INSERT INTO projects (title, budget, dept_id)
VALUES ('Website', 5000, 1), ('Tuyển dụng', 2000, 2), ('Quảng cáo', 8000, 3);

SELECT * FROM projects;
```

KẾT QUẢ · 3 dòng

project_id	title	budget	status	dept_id
1	Website	5000	new	1
2	Tuyển dụng	2000	new	2
3	Quảng cáo	8000	new	3

3.5 UPDATE – sửa dữ liệu

Luôn có `WHERE`, nếu không mọi dòng của bảng sẽ bị sửa.

CÚ PHÁP

```
UPDATE table_name
SET col1 = value1, col2 = value2
WHERE condition;
```

VÍ DỤ

```
UPDATE projects
SET budget = budget + 500, status = 'active'
WHERE title = 'Website';

SELECT * FROM projects;
```

KẾT QUẢ · 3 dòng

project_id	title	budget	status	dept_id
1	Website	5500	active	1
2	Tuyển dụng	2000	new	2
3	Quảng cáo	8000	new	3

3.6 DELETE – xóa dữ liệu

Cũng nên có `WHERE`. Nên chạy `SELECT` với cùng điều kiện trước để kiểm tra các dòng sẽ bị xóa.

CÚ PHÁP

```
DELETE FROM table_name
WHERE condition;
```

VÍ DỤ

```
DELETE FROM projects
WHERE budget < 3000;

SELECT * FROM projects;
```

KẾT QUẢ · 2 dòng

project_id	title	budget	status	dept_id
1	Website	5500	active	1
3	Quảng cáo	8000	new	3

3.7 ALTER TABLE – đổi cấu trúc bảng

CÚ PHÁP

```
ALTER TABLE t ADD COLUMN col datatype;
ALTER TABLE t RENAME COLUMN old_name TO new_name;
ALTER TABLE t DROP COLUMN col;
```

VÍ DỤ

```
ALTER TABLE projects ADD COLUMN owner TEXT;
ALTER TABLE projects RENAME COLUMN owner TO manager;

SELECT * FROM projects;
```

KẾT QUẢ · 2 dòng

project_id	title	budget	status	dept_id	manager
1	Website	5500	active	1	*NULL*
3	Quảng cáo	8000	new	3	*NULL*

Lưu ý: Cú pháp sửa kiểu cột khác nhau: PostgreSQL `ALTER COLUMN c TYPE ...`, MySQL `MODIFY COLUMN ...`, SQL Server `ALTER COLUMN ...`.

3.8 DROP TABLE và TRUNCATE

`DROP TABLE` xóa cả bảng. `TRUNCATE` xóa sạch dữ liệu nhanh nhưng giữ cấu trúc (SQLite không có, dùng `DELETE FROM t`). Cả hai đều **không thể hoàn tác** ở nhiều hệ CSDL.

CÚ PHÁP

```
DROP TABLE [IF EXISTS] table_name;
TRUNCATE TABLE table_name;   -- MySQL, PostgreSQL, SQL Server, Oracle
```

VÍ DỤ

```
DROP TABLE IF EXISTS projects;
```

✓ Thực thi thành công

4. JOIN – ghép nhiều bảng

`JOIN` ghép dòng của hai bảng dựa trên điều kiện `ON`. Chọn loại JOIN tùy việc có muốn giữ lại dòng không khớp hay không.

CÚ PHÁP

```
SELECT ...
FROM table_a a
[INNER | LEFT | RIGHT | FULL] JOIN table_b b ON a.key = b.key;
```

Loại JOIN	Kết quả
`INNER JOIN`	Chỉ các dòng khớp ở **cả hai** bảng
`LEFT JOIN`	Mọi dòng bảng trái; không khớp thì cột bảng phải là NULL
`RIGHT JOIN`	Mọi dòng bảng phải; không khớp thì cột bảng trái là NULL
`FULL JOIN`	Mọi dòng của cả hai bảng (MySQL không hỗ trợ)
`CROSS JOIN`	Tích Descartes: mọi cặp (dòng A, dòng B), không có `ON`
Self join	Một bảng ghép với chính nó (dùng alias khác nhau)

4.1 INNER JOIN

Nhân viên Khoa (không có phòng ban) và phòng Marketing (không có nhân viên) đều không xuất hiện.

VÍ DỤ

```
SELECT e.name, d.dept_name, d.location
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

KẾT QUẢ · 8 dòng

name	dept_name	location
An	IT	Hà Nội
Bình	IT	Hà Nội
Chi	IT	Hà Nội
Dũng	HR	TP.HCM
Em	Sales	Đà Nẵng
Phúc	Sales	Đà Nẵng
Giang	Sales	Đà Nẵng
Hà	HR	TP.HCM

4.2 LEFT JOIN

Giữ toàn bộ phòng ban, kể cả phòng chưa có nhân viên.

VÍ DỤ

```sql
SELECT d.dept_name, e.name
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
ORDER BY d.dept_id;
```

KẾT QUẢ · 9 dòng

dept_name	name
IT	An
IT	Bình
IT	Chi
HR	Dũng
HR	Hà
Sales	Em
Sales	Giang
Sales	Phúc
Marketing	*NULL*

4.3 Anti-join: tìm dòng không có đối tượng khớp

`LEFT JOIN` + `WHERE … IS NULL` để tìm phòng ban không có nhân viên nào.

VÍ DỤ

```sql
SELECT d.dept_name
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

KẾT QUẢ · 1 dòng

dept_name
Marketing

4.4 FULL OUTER JOIN

Lấy các dòng không khớp ở **bất kỳ phía nào**.

VÍ DỤ

```sql
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL OR d.dept_id IS NULL;
```

KẾT QUẢ · 2 dòng

name	dept_name
Khoa	*NULL*
NULL	Marketing

Lưu ý: MySQL chưa có `FULL JOIN`: dùng `LEFT JOIN ... UNION ... RIGHT JOIN ...`.

4.5 CROSS JOIN

Mọi tổ hợp giữa hai tập dòng (ví dụ: tạo lưới phòng ban × trạng thái).

VÍ DỤ

```sql
SELECT d.dept_name, s.status
FROM departments d
CROSS JOIN (SELECT 'open' AS status UNION ALL SELECT 'closed') s
WHERE d.dept_id <= 2;
```

KẾT QUẢ · 4 dòng

dept_name	status
IT	open
IT	closed
HR	open
HR	closed

4.6 Self join

Tìm quản lý của từng nhân viên bằng cách ghép bảng `employees` với chính nó.

VÍ DỤ

```sql
SELECT e.name AS nhan_vien, m.name AS quan_ly
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id
WHERE e.dept_id = 3;
```

KẾT QUẢ · 3 dòng

nhan_vien	quan_ly
Em	An
Phúc	Em
Giang	Em

4.7 JOIN + GROUP BY – báo cáo doanh thu

`USING (col)` là cách viết gọn khi hai bảng có cùng tên cột khóa.

VÍ DỤ

```sql
SELECT p.category, SUM(o.qty * p.price) AS doanh_thu
FROM orders o
JOIN products p USING (product_id)
WHERE o.status <> 'cancelled'
GROUP BY p.category
ORDER BY doanh_thu DESC;
```

KẾT QUẢ · 2 dòng

category	doanh_thu
Electronics	3925
Furniture	1200

5. Truy vấn con (Subquery)

5.1 Subquery trả về một giá trị (scalar)

Dùng kết quả của câu truy vấn con như một giá trị trong biểu thức so sánh.

CÚ PHÁP

```
SELECT ... FROM t
WHERE col > (SELECT AGG(col) FROM t);
```

VÍ DỤ

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

KẾT QUẢ · 4 dòng

name	salary
An	3000
Bình	2200
Chi	2000
Em	2500

5.2 IN (subquery)

Khách hàng đã mua sản phẩm thuộc nhóm Furniture.

VÍ DỤ

```
SELECT DISTINCT customer
FROM orders
WHERE product_id IN (SELECT product_id FROM products WHERE category = 'Furniture');
```

KẾT QUẢ · 4 dòng

customer
Carol
Dave
Alice
Bob

5.3 EXISTS

Trả về TRUE nếu subquery có ít nhất một dòng; thường nhanh với bảng lớn. `NOT EXISTS` là phép phủ định.

CÚ PHÁP

```
WHERE [NOT] EXISTS (SELECT 1 FROM other WHERE other.key = outer.key)
```

VÍ DỤ

```
SELECT dept_name
FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

KẾT QUẢ · 3 dòng

dept_name
IT
HR
Sales

5.4 Correlated subquery

Subquery tham chiếu cột của truy vấn ngoài, nên được tính lại cho mỗi dòng. Ví dụ: nhân viên lương cao hơn trung bình phòng mình.

VÍ DỤ

```sql
SELECT name, dept_id, salary
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);
```

KẾT QUẢ · 3 dòng

name	dept_id	salary
An	1	3000
Dũng	2	1800
Em	3	2500

5.5 Subquery trong FROM (derived table)

Kết quả subquery được coi như một bảng tạm, bắt buộc phải có alias.

VÍ DỤ

```sql
SELECT dept_id, ROUND(avg_sal) AS luong_tb
FROM (SELECT dept_id, AVG(salary) AS avg_sal
      FROM employees GROUP BY dept_id) t
WHERE avg_sal > 1700;
```

KẾT QUẢ · 2 dòng

dept_id	luong_tb
1	2400.0
3	1867.0

6. Phép toán tập hợp

Hai truy vấn phải có **cùng số cột** và kiểu dữ liệu tương thích. `UNION` loại bỏ dòng trùng; `UNION ALL` giữ nguyên (nhanh hơn).

CÚ PHÁP

```sql
SELECT ... FROM a
UNION [ALL] | INTERSECT | EXCEPT
SELECT ... FROM b;
```

6.1 UNION

Khách có đơn đang chờ hoặc đơn đã hủy.

VÍ DỤ

```sql
SELECT customer FROM orders WHERE status = 'pending'
UNION
SELECT customer FROM orders WHERE status = 'cancelled';
```

KẾT QUẢ · 3 dòng

customer
Alice
Carol
Dave

6.2 UNION ALL

Giữ cả dòng trùng (Carol xuất hiện hai lần).

VÍ DỤ

```sql
SELECT customer FROM orders WHERE status = 'pending'
UNION ALL
SELECT customer FROM orders WHERE qty >= 3;
```

KẾT QUẢ · 5 dòng

customer
Carol
Alice
Bob
Dave
Carol

6.3 INTERSECT

Khách vừa có đơn hoàn tất vừa có đơn đang chờ.

VÍ DỤ

```sql
SELECT customer FROM orders WHERE status = 'done'
INTERSECT
SELECT customer FROM orders WHERE status = 'pending';
```

KẾT QUẢ · 2 dòng

customer
Alice
Carol

6.4 EXCEPT

Khách có đơn hoàn tất nhưng chưa từng có đơn đang chờ (Oracle dùng `MINUS`).

VÍ DỤ

```sql
SELECT customer FROM orders WHERE status = 'done'
EXCEPT
SELECT customer FROM orders WHERE status = 'pending';
```

KẾT QUẢ · 2 dòng

customer
Bob
Eve

7. Hàm & biểu thức

7.1 Hàm chuỗi

`||` là toán tử nối chuỗi của chuẩn SQL (MySQL dùng `CONCAT()`, SQL Server dùng `+`).

CÚ PHÁP

```
UPPER(s)  LOWER(s)  LENGTH(s)  SUBSTR(s, start, len)
TRIM(s)  REPLACE(s, from, to)  s1 || s2
```

VÍ DỤ

```sql
SELECT UPPER(name) AS hoa, LENGTH(name) AS do_dai,
       SUBSTR(job_title, 1, 5) AS cat,
       REPLACE(job_title, 'Sales', 'KD') AS thay,
       name || ' - ' || job_title AS gop
FROM employees
WHERE emp_id = 6;
```

KẾT QUẢ · 1 dòng

hoa	do_dai	cat	thay	gop
PHúC	4	Sales	KD Rep	Phúc - Sales Rep

Lưu ý: `UPPER` / `LOWER` của SQLite chỉ đổi hoa/thường chữ ASCII nên "Phúc" thành "PHúC". MySQL, PostgreSQL, SQL Server, Oracle xử lý đúng chữ có dấu (tùy collation).

7.2 Hàm số

Chú ý phép chia hai số nguyên: nhiều hệ CSDL trả về số nguyên (7/2 = 3).

CÚ PHÁP

```
ROUND(x, n)  ABS(x)  x % y   -- chia lấy dư
```

VÍ DỤ

```sql
SELECT ROUND(3.14159, 2) AS lam_tron, ABS(-7) AS tri_tuyet_doi,
       17 % 5 AS chia_du, 7 / 2 AS chia_nguyen, 7 / 2.0 AS chia_thuc;
```

KẾT QUẢ · 1 dòng

lam_tron	tri_tuyet_doi	chia_du	chia_nguyen	chia_thuc
3.14	7	2	3	3.5

7.3 CASE – rẽ nhánh

Giống `if / else` trong SQL; có thể dùng trong `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`.

CÚ PHÁP

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE default_result
END
```

VÍ DỤ

```
SELECT name, salary,
       CASE WHEN salary >= 2500 THEN 'Cao'
            WHEN salary >= 1700 THEN 'Trung bình'
            ELSE 'Thấp' END AS bac_luong
FROM employees
LIMIT 5;
```

KẾT QUẢ · 5 dòng

name	salary	bac_luong
An	3000	Cao
Bình	2200	Trung bình
Chi	2000	Trung bình
Dũng	1800	Trung bình
Em	2500	Cao

7.4 COALESCE, NULLIF

`COALESCE` trả về giá trị đầu tiên khác NULL. `NULLIF(a, b)` trả về NULL nếu a = b (hay dùng để tránh chia cho 0).

CÚ PHÁP

```
COALESCE(v1, v2, ..., default)
NULLIF(a, b)
```

VÍ DỤ

```
SELECT name, COALESCE(dept_id, 0) AS dept, NULLIF(job_title, 'Intern') AS chuc_vu
FROM employees
WHERE emp_id IN (1, 9);
```

KẾT QUẢ · 2 dòng

name	dept	chuc_vu
An	1	Director
Khoa	0	*NULL*

7.5 CAST – ép kiểu

CÚ PHÁP

```
CAST(expression AS datatype)
```

VÍ DỤ

```
SELECT CAST('123' AS INTEGER) + 1 AS so, CAST(7 AS REAL) / 2 AS thuc,
       CAST(3.99 AS INTEGER) AS cat_phan_le;
```

KẾT QUẢ · 1 dòng

so	thuc	cat_phan_le
124	3.5	3

7.6 Ngày & giờ

Hàm ngày giờ khác nhau nhiều giữa các hệ CSDL (xem bảng bên dưới). Ví dụ chạy trên SQLite:

Mục đích	SQLite	MySQL	PostgreSQL	SQL Server
Ngày hiện tại	`date('now')`	`CURDATE()`	`CURRENT_DATE`	`CAST(GETDATE() AS DATE)`
Cộng 1 tháng	`date(d,'+1 month')`	`DATE_ADD(d, INTERVAL 1 MONTH)`	`d + INTERVAL '1 month'`	`DATEADD(month, 1, d)`
Chênh lệch ngày	`julianday(a)-julianday(b)`	`DATEDIFF(a, b)`	`a - b`	`DATEDIFF(day, b, a)`
Lấy năm	`strftime('%Y', d)`	`YEAR(d)`	`EXTRACT(YEAR FROM d)`	`YEAR(d)`
Định dạng	`strftime('%d/%m/%Y', d)`	`DATE_FORMAT(d, '%d/%m/%Y')`	`TO_CHAR(d, 'DD/MM/YYYY')`	`FORMAT(d, 'dd/MM/yyyy')`

VÍ DỤ

```sql
SELECT name, hire_date,
       strftime('%Y', hire_date) AS nam,
       date(hire_date, '+1 year') AS sau_1_nam,
       CAST(julianday('2026-10-12') - julianday(hire_date) AS INTEGER) AS so_ngay
FROM employees
LIMIT 3;
```

KẾT QUẢ · 3 dòng

name	hire_date	nam	sau_1_nam	so_ngay
An	2019-03-01	2019	2020-03-01	2782
Bình	2020-06-15	2020	2021-06-15	2310
Chi	2021-01-10	2021	2022-01-10	2101

8. Nâng cao

8.1 CTE – WITH

Common Table Expression: đặt tên cho một truy vấn tạm để dùng lại, giúp câu lệnh phức tạp dễ đọc hơn so với subquery lồng nhau.

CÚ PHÁP

```sql
WITH cte_name AS (
  SELECT ...
)
SELECT ... FROM cte_name;
```

VÍ DỤ

```sql
WITH dept_stats AS (
  SELECT dept_id, AVG(salary) AS avg_sal
  FROM employees GROUP BY dept_id
)
SELECT e.name, e.salary, ROUND(s.avg_sal) AS tb_phong
FROM employees e
JOIN dept_stats s ON e.dept_id = s.dept_id
WHERE e.salary > s.avg_sal;
```

KẾT QUẢ · 3 dòng

name	salary	tb_phong
An	3000	2400.0
Dũng	1800	1600.0
Em	2500	1867.0

8.2 Recursive CTE – dữ liệu phân cấp

Duyệt cây (sơ đồ tổ chức, danh mục cha–con): phần neo (anchor) + phần đệ quy nối bằng `UNION ALL`.

CÚ PHÁP

```
WITH RECURSIVE t(cols) AS (
  SELECT ...            -- anchor
  UNION ALL
  SELECT ... FROM t JOIN ...   -- recursive step
)
SELECT * FROM t;
```

VÍ DỤ

```
WITH RECURSIVE org(emp_id, name, lvl) AS (
  SELECT emp_id, name, 0 FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.name, o.lvl + 1
  FROM employees e JOIN org o ON e.manager_id = o.emp_id
)
SELECT lvl, name FROM org ORDER BY lvl, name;
```

KẾT QUẢ · 9 dòng

lvl	name
0	An
1	Bình
1	Chi
1	Dũng
1	Em
1	Khoa
2	Giang
2	Hà
2	Phúc

8.3 Window function: ROW_NUMBER, RANK, DENSE_RANK

Tính toán trên một "cửa sổ" các dòng **mà không gộp dòng** như `GROUP BY`. `PARTITION BY` chia nhóm, `ORDER BY` xác định thứ tự trong nhóm.

CÚ PHÁP

```
function() OVER (
  [PARTITION BY col1, ...]
  [ORDER BY col2 [ASC|DESC]]
  [ROWS BETWEEN ... AND ...]
)
```

VÍ DỤ

```
SELECT name, dept_id, salary,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn,
       RANK()       OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rk
FROM employees
WHERE dept_id IS NOT NULL;
```

KẾT QUẢ · 8 dòng

name	dept_id	salary	rn	rk
An	1	3000	1	1
Bình	1	2200	2	2
Chi	1	2000	3	3
Dũng	2	1800	1	1
Hà	2	1400	2	2
Em	3	2500	1	1
Giang	3	1600	2	2
Phúc	3	1500	3	3

8.4 Top-N mỗi nhóm

Mẫu rất hay gặp: đánh số thứ hạng bằng window function trong CTE rồi lọc.

VÍ DỤ

```
WITH r AS (
  SELECT name, dept_id, salary,
         ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
  FROM employees WHERE dept_id IS NOT NULL
)
SELECT name, dept_id, salary FROM r WHERE rn = 1;
```

KẾT QUẢ · 3 dòng

name	dept_id	salary
An	1	3000
Dũng	2	1800
Em	3	2500

8.5 LAG / LEAD và tổng lũy kế

`LAG` lấy giá trị dòng trước, `LEAD` lấy dòng sau. `SUM() OVER (ORDER BY ...)` cho tổng cộng dồn (running total).

VÍ DỤ

```sql
SELECT order_date, qty,
       LAG(qty)  OVER (ORDER BY order_date) AS qty_truoc,
       SUM(qty)  OVER (ORDER BY order_date) AS luy_ke,
       ROUND(AVG(qty) OVER (ORDER BY order_date
             ROWS BETWEEN 1 PRECEDING AND CURRENT ROW), 1) AS tb_2_don
FROM orders
ORDER BY order_date
LIMIT 6;
```

KẾT QUẢ · 6 dòng

order_date	qty	qty_truoc	luy_ke	tb_2_don
2026-01-05	1	*NULL*	1	1.0
2026-01-12	3	1	4	2.0
2026-02-03	2	3	6	2.5
2026-02-14	1	2	7	1.5
2026-03-01	1	1	8	1.0
2026-03-09	4	1	12	2.5

8.6 NTILE và bảng tóm tắt window function

`NTILE(n)` chia các dòng thành n nhóm đều nhau (ví dụ: chia lương thành 3 bậc).

Hàm	Công dụng
`ROW_NUMBER()`	Số thứ tự liên tục 1, 2, 3, ... (không trùng)
`RANK()`	Hạng, nếu đồng hạng sẽ **nhảy số**: 1, 1, 3
`DENSE_RANK()`	Hạng, đồng hạng **không nhảy số**: 1, 1, 2
`NTILE(n)`	Chia thành n nhóm
`LAG(c, k)` / `LEAD(c, k)`	Giá trị của dòng trước / sau k dòng
`FIRST_VALUE(c)` / `LAST_VALUE(c)`	Giá trị đầu / cuối của khung
`SUM / AVG / COUNT ... OVER`	Hàm tổng hợp dạng cửa sổ (tổng lũy kế, trung bình trượt)

VÍ DỤ

```sql
SELECT name, salary, NTILE(3) OVER (ORDER BY salary DESC) AS bac
FROM employees;
```

KẾT QUẢ · 9 dòng

name	salary	bac
An	3000	1
Em	2500	1
Bình	2200	1
Chi	2000	2
Dũng	1800	2
Khoa	1700	2
Giang	1600	3
Phúc	1500	3
Hà	1400	3

8.7 Pivot bằng CASE

Xoay giá trị của một cột thành nhiều cột bằng `SUM(CASE WHEN ...)`.

VÍ DỤ

```sql
SELECT customer,
       SUM(CASE WHEN status = 'done'      THEN qty ELSE 0 END) AS done,
       SUM(CASE WHEN status = 'pending'   THEN qty ELSE 0 END) AS pending,
       SUM(CASE WHEN status = 'cancelled' THEN qty ELSE 0 END) AS cancelled
FROM orders
GROUP BY customer;
```

KẾT QUẢ · 5 dòng

customer	done	pending	cancelled
Alice	3	2	0
Bob	6	0	0
Carol	10	1	0
Dave	0	0	4
Eve	1	0	0

8.8 VIEW – bảng ảo

Lưu một câu `SELECT` dưới dạng tên để dùng lại như một bảng; không lưu dữ liệu riêng.

CÚ PHÁP

```sql
CREATE VIEW view_name AS
SELECT ...;
DROP VIEW view_name;
```

VÍ DỤ

```sql
CREATE VIEW v_emp_dept AS
SELECT e.name, d.dept_name, e.salary
FROM employees e JOIN departments d ON e.dept_id = d.dept_id;

SELECT * FROM v_emp_dept WHERE salary > 2000;
```

KẾT QUẢ · 3 dòng

name	dept_name	salary
An	IT	3000
Bình	IT	2200
Em	Sales	2500

8.9 INDEX – chỉ mục

Giúp tìm kiếm / JOIN / sắp xếp nhanh hơn nhưng làm chậm ghi và tốn dung lượng. Nên đặt trên cột hay dùng ở `WHERE`, `JOIN`, `ORDER BY`.

CÚ PHÁP

```sql
CREATE [UNIQUE] INDEX index_name ON table_name (col1, col2);
DROP INDEX index_name;
```

VÍ DỤ

```sql
CREATE INDEX idx_emp_dept ON employees(dept_id);

EXPLAIN QUERY PLAN
SELECT * FROM employees WHERE dept_id = 3;
```

KẾT QUẢ · 1 dòng

id	parent	notused	detail
3	0	0	SEARCH employees USING INDEX idx_emp_dept (dept_id=?)

Lưu ý: Dùng `EXPLAIN` (MySQL, PostgreSQL) hoặc `EXPLAIN QUERY PLAN` (SQLite) để xem câu lệnh có dùng chỉ mục hay không.

8.10 Transaction – giao dịch

Nhóm nhiều lệnh thành một đơn vị: hoặc tất cả cùng thành công (`COMMIT`) hoặc hủy toàn bộ (`ROLLBACK`). Đảm bảo tính ACID.

CÚ PHÁP

```
BEGIN;            -- hoặc START TRANSACTION
  ... các lệnh INSERT / UPDATE / DELETE ...
COMMIT;           -- lưu thay đổi
-- hoặc ROLLBACK;  -- hủy thay đổi
```

VÍ DỤ

```
BEGIN;
UPDATE employees SET salary = 0;     -- lỡ tay!
ROLLBACK;

SELECT MIN(salary) AS luong_thap_nhat FROM employees;
```

KẾT QUẢ · 1 dòng

luong_thap_nhat
1400

8.11 UPSERT – thêm hoặc cập nhật

Nếu khóa đã tồn tại thì cập nhật, ngược lại thì thêm mới.

CÚ PHÁP

```
INSERT INTO t (key, val) VALUES (...)
ON CONFLICT (key) DO UPDATE SET val = excluded.val;   -- PostgreSQL, SQLite
-- MySQL: INSERT ... ON DUPLICATE KEY UPDATE val = VALUES(val);
-- SQL Server / Oracle: MERGE INTO ... USING ... WHEN MATCHED THEN ...
```

VÍ DỤ

```
CREATE TABLE stock (product_id INTEGER PRIMARY KEY, qty INTEGER);
INSERT INTO stock VALUES (1, 10);

INSERT INTO stock VALUES (1, 5)
ON CONFLICT (product_id) DO UPDATE SET qty = qty + excluded.qty;

SELECT * FROM stock;
```

KẾT QUẢ · 1 dòng

product_id	qty
1	15

8.12 TRIGGER

Đoạn lệnh tự chạy khi có `INSERT` / `UPDATE` / `DELETE`. Ví dụ ghi log mỗi khi lương thay đổi (`OLD` = giá trị cũ, `NEW` = giá trị mới).

CÚ PHÁP

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE} ON table_name
FOR EACH ROW
BEGIN
  ...
END;
```

VÍ DỤ

```
CREATE TABLE salary_log (emp_id INTEGER, old_salary INTEGER, new_salary INTEGER);

CREATE TRIGGER trg_salary AFTER UPDATE OF salary ON employees
BEGIN
  INSERT INTO salary_log VALUES (OLD.emp_id, OLD.salary, NEW.salary);
END;

UPDATE employees SET salary = 2300 WHERE emp_id = 2;
SELECT * FROM salary_log;
```

KẾT QUẢ · 1 dòng

emp_id	old_salary	new_salary
2	2200	2300

Lưu ý: Cú pháp trigger khác nhau khá nhiều giữa các hệ CSDL (PostgreSQL cần tạo function riêng, MySQL dùng `DELIMITER`).

9. Mẫu truy vấn hay gặp

9.1 Tìm bản ghi trùng

Gom nhóm theo cột nghi trùng và lọc `HAVING COUNT(*) > 1`. Ví dụ: khách hàng có nhiều hơn 1 đơn.

VÍ DỤ

```
SELECT customer, COUNT(*) AS so_don
FROM orders
GROUP BY customer
HAVING COUNT(*) > 1;
```

KẾT QUẢ · 3 dòng

customer	so_don
Alice	3
Bob	3
Carol	2

9.2 Giá trị lớn thứ N

Lương cao thứ hai – hai cách phổ biến.

VÍ DỤ

```
-- Cách 1: subquery
SELECT MAX(salary) AS luong_cao_thu_2
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Cách 2: LIMIT / OFFSET
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

KẾT QUẢ · 1 dòng

salary
2500

9.3 Tỷ lệ phần trăm trên tổng

`SUM(...) OVER ()` (cửa sổ rỗng) cho tổng toàn bộ để tính phần trăm đóng góp.

VÍ DỤ

```sql
SELECT p.category, SUM(o.qty * p.price) AS doanh_thu,
       ROUND(100.0 * SUM(o.qty * p.price)
             / SUM(SUM(o.qty * p.price)) OVER (), 1) AS ty_le_pct
FROM orders o JOIN products p USING (product_id)
WHERE o.status <> 'cancelled'
GROUP BY p.category;
```

KẾT QUẢ · 2 dòng

category	doanh_thu	ty_le_pct
Electronics	3925	76.6
Furniture	1200	23.4

9.4 Khách chưa từng hủy đơn

`NOT EXISTS` hoặc `NOT IN` (cẩn thận `NOT IN` với NULL).

VÍ DỤ

```sql
SELECT DISTINCT o.customer
FROM orders o
WHERE NOT EXISTS (
  SELECT 1 FROM orders c
  WHERE c.customer = o.customer AND c.status = 'cancelled'
);
```

KẾT QUẢ · 4 dòng

customer
Alice
Bob
Carol
Eve

9.5 Doanh thu theo tháng

Rút tháng từ ngày rồi gom nhóm.

VÍ DỤ

```sql
SELECT strftime('%Y-%m', o.order_date) AS thang,
       SUM(o.qty * p.price) AS doanh_thu
FROM orders o JOIN products p USING (product_id)
WHERE o.status = 'done'
GROUP BY thang
ORDER BY thang;
```

KẾT QUẢ · 5 dòng

thang	doanh_thu
2026-01	1275
2026-02	800
2026-03	1450
2026-04	400
2026-05	600

10. Phụ lục

A. Khác biệt giữa các hệ CSDL phổ biến

SQL có chuẩn ANSI nhưng mỗi hệ CSDL có thêm "phương ngữ" riêng. Bảng dưới liệt kê những điểm hay gây lỗi khi chuyển đổi.

Chức năng	MySQL / PostgreSQL / SQLite	SQL Server	Oracle
Giới hạn số dòng	`LIMIT n [OFFSET m]`	`SELECT TOP n ...` hoặc `OFFSET m ROWS FETCH NEXT n ROWS ONLY`	`FETCH FIRST n ROWS ONLY` (12c+)
Nối chuỗi	`a || b` (PostgreSQL, SQLite); `CONCAT(a, b)` (MySQL)	`a + b` hoặc `CONCAT()`	`a || b`
Thời điểm hiện tại	`NOW()`, `CURRENT_TIMESTAMP`	`GETDATE()`	`SYSDATE`
Cột tự tăng	`AUTO_INCREMENT` (MySQL); `SERIAL` / `GENERATED ... AS IDENTITY` (PostgreSQL); `INTEGER PRIMARY KEY` (SQLite)	`IDENTITY(1,1)`	`GENERATED AS IDENTITY`
Thay giá trị NULL	`COALESCE`; `IFNULL` (MySQL, SQLite)	`ISNULL`, `COALESCE`	`NVL`, `COALESCE`
Upsert	`ON DUPLICATE KEY UPDATE` (MySQL); `ON CONFLICT DO UPDATE` (PostgreSQL, SQLite)	`MERGE`	`MERGE`
FULL OUTER JOIN	PostgreSQL, SQLite (3.39+) có; **MySQL không**	Có	Có
LIKE không phân biệt hoa/thường	`ILIKE` (PostgreSQL); MySQL mặc định không phân biệt	Tùy collation	`UPPER(a) LIKE UPPER(b)`
Phép hiệu hai tập	`EXCEPT` (MySQL 8.0.31+)	`EXCEPT`	`MINUS`

B. Thói quen tốt khi viết SQL

• **Liệt kê cột cần dùng** thay vì `SELECT *`, đặc biệt trong code ứng dụng.
• **Luôn có** `WHERE` **cho** `UPDATE` **/** `DELETE`; chạy thử bằng `SELECT` cùng điều kiện, và bọc trong transaction khi làm việc quan trọng.
• **Đặt alias ngắn, rõ nghĩa** cho bảng khi JOIN (`e`, `d`, `o`) và ghi rõ `table.column` để tránh nhập nhằng.
• **Cẩn thận NULL**: `NULL = NULL` không phải TRUE; `NOT IN (subquery)` trả về rỗng nếu subquery có NULL – ưu tiên `NOT EXISTS`.
• **Tạo chỉ mục** cho cột dùng nhiều ở `WHERE` / `JOIN` / `ORDER BY`; kiểm tra bằng `EXPLAIN`.
• Dùng **CTE** để chia câu lệnh dài thành các bước dễ đọc; thụt dòng và viết hoa từ khóa nhất quán.
• **Tham số hóa truy vấn** (prepared statement) thay vì nối chuỗi từ input của người dùng để tránh SQL injection.